🚀 JavaScript - Preguntas Frecuentes en Entrevistas

Este documento contiene una recopilación de preguntas comunes sobre JavaScript en entrevistas técnicas, con respuestas detalladas y ejemplos prácticos.

🔹 **Cada pregunta está explicada en detalle en este mismo archivo.**
Puedes recorrerlo fácilmente usando el índice. 🚀

📌 Índice

1. ¿Cuáles son los distintos tipos de datos en JavaScript?
2. ¿Cuál es la diferencia entre los operadores `==` y `===`?
3. ¿Qué es la coerción implícita de tipos en JavaScript?
4. ¿Es JavaScript un lenguaje de tipo estático o dinámico?
5. ¿Qué es la propiedad `NaN` en JavaScript?
6. ¿Qué es el paso por valor y el paso por referencia?
7. ¿Qué son las funciones de orden superior en JavaScript?
8. Explica los métodos `call()`, `apply()` y `bind()`
9. ¿Qué es Currying en JavaScript?
10. ¿Qué son las Closures en JavaScript?
11. Diferencias entre `var`, `let` y `const`
12. ¿Para qué sirven las promesas en JavaScript?
13. ¿Qué son las clases en JavaScript?
14. ¿Qué es la destructuración de objetos?
15. ¿Qué son las funciones generadoras?
16. Explica el alcance y la cadena de alcance en JavaScript.
17. ¿Qué son las devoluciones de llamada (Callbacks) en JavaScript?
18. ¿Cuál es la diferencia entre declaración de función y expresión de función?
19. ¿Qué son las importaciones y exportaciones en JavaScript?
20. Explica qué es el DOM y cómo funciona.
21. ¿Qué es el parámetro `rest` y el operador `spread`?
22. ¿Cuál es la diferencia entre `Undefined`, `Undeclared` y `Null` en JavaScript?
23. Recursos adicionales

📌 Preguntas y Respuestas

🔹 ¿Cuáles son los distintos tipos de datos en JavaScript?

JavaScript tiene **7 tipos de datos primitivos** y tipos estructurales:

Tipo	Ejemplo

`String`	`"Hola, mundo"`
`Number`	`42`, `3.14`
`Boolean`	`true`, `false`
`Undefined`	`let x;`
`Null`	`let y = null;`
`Symbol`	`const sym = Symbol("id");`
`BigInt`	`const big = 12345678901234567890n;`

También existen tipos estructurales como **Objects, Arrays y Functions**.

🔹 ¿Cuál es la diferencia entre los operadores `==` y `===`?

Operador	Comparación	Ejemplo	Resultado
`==`	Compara valores, permite conversión de tipos (coerción)	`5 == "5"`	`true`
`===`	Compara valores y tipos estrictamente	`5 === "5"`	`false`

🔹 **Ejemplo:**

```js
console.log(0 == false);  // true (coerción)
console.log(0 === false); // false (sin coerción)
```

🔹 ¿Qué es la coerción implícita de tipos en JavaScript?

Es cuando JavaScript convierte automáticamente un tipo de dato en otro.

🔹 **Ejemplo de coerción:**

```js
console.log("5" + 5); // "55" (Number se convierte en String)
console.log("5" - 3); // 2 (String se convierte en Number)
```

🔹 ¿Es JavaScript un lenguaje de tipo estático o dinámico?

JavaScript es **dinámico**, lo que significa que las variables pueden cambiar de tipo en tiempo de ejecución.

```js
let x = "Hola"; // String
x = 10;         // Ahora es Number
```

🔹 ¿Qué es la propiedad `NaN` en JavaScript?

`NaN` (Not a Number) representa un valor que no es un número válido.

🔹 **Ejemplo:**

```js
console.log(0 / 0);       // NaN
console.log(Math.sqrt(-1)); // NaN
console.log(isNaN("abc")); // true
```

🔹 ¿Qué es el paso por valor y el paso por referencia?

Modo	Tipos de datos	¿Se modifica el original?
Paso por valor	Primitivos (`String`, `Number`, etc.)	❌ No
Paso por referencia	Objetos y Arrays	✅ Sí

🔹 **Ejemplo:**

```js
let a = 10;
let b = a; // Se copia el valor
b = 20;
console.log(a); // 10

let obj1 = { name: "Juan" };
let obj2 = obj1; // Ambas referencias apuntan al mismo objeto
obj2.name = "Pedro";
console.log(obj1.name); // "Pedro"
```

🔹 ¿Qué son las funciones de orden superior en JavaScript?

Son funciones que **reciben o retornan otra función**.

🔹 **Ejemplo:**

```js
function operacion(a, b, callback) {
  return callback(a, b);
}

const suma = (x, y) => x + y;
console.log(operacion(5, 3, suma)); // 8
```

🔹 Explica los métodos `call()`, `apply()` y `bind()`

Son métodos para cambiar el contexto de `this` en funciones.

Método	Uso	Diferencia
`call()`	Ejecuta la función con `this` modificado y **parámetros separados**	`func.call(obj, arg1, arg2)`
`apply()`	Igual que `call()`, pero **parámetros en un array**	`func.apply(obj, [arg1, arg2])`
`bind()`	Devuelve una nueva función con `this` cambiado	`const newFunc = func.bind(obj)`

🔹 **Ejemplo:**

```js
const persona = { nombre: "Carlos" };
function saludar() {
  console.log("Hola " + this.nombre);
}
saludar.call(persona); // "Hola Carlos"
```

🔹 ¿Qué es Currying en JavaScript?

Currying es una técnica donde una función recibe **uno a uno sus argumentos** en lugar de todos a la vez.

🔹 **Ejemplo:**

```js
const suma = a => b => a + b;
console.log(suma(3)(5)); // 8
```

🔹 ¿Qué son las Closures en JavaScript?

Una **Closure** es una función que mantiene acceso a las variables de su contexto incluso después de ejecutarse.

🔹 **Ejemplo:**

```js
function contador() {
  let count = 0;
  return () => count++;
}
const increment = contador();
console.log(increment()); // 1
console.log(increment()); // 2
```

🔹 Diferencias entre `var`, `let` y `const`

Declaración	Ámbito	Hoisting	Reasignable
`var`	Función	Sí, como `undefined`	Sí
`let`	Bloque	Sí, pero no inicializado	Sí
`const`	Bloque	Sí, pero no inicializado	No

🔹 ¿Para qué sirven las promesas en JavaScript?

Las **promesas** manejan operaciones asíncronas y evitan el "callback hell".

🔹 **Ejemplo:**

```js
const promesa = new Promise((resolve) => setTimeout(() => resolve("Completado"),
```

```
2000));
promesa.then(console.log);
```

🔹 ¿Qué son las clases en JavaScript?

Las clases son una forma de definir objetos en JavaScript usando `class`.

🔹 **Ejemplo:**

```js
class Persona {
  constructor(nombre) {
    this.nombre = nombre;
  }
  saludar() {
    return `Hola, soy ${this.nombre}`;
  }
}
const juan = new Persona("Juan");
console.log(juan.saludar());
```

🔹 ¿Qué es la destructuración de objetos?

Es una forma de extraer valores de objetos o arrays.

🔹 **Ejemplo:**

```js
const usuario = { nombre: "Ana", edad: 25 };
const { nombre, edad } = usuario;
console.log(nombre, edad); // "Ana 25"
```

🔹 ¿Qué son las funciones generadoras?

Las **funciones generadoras** (`function*`) son un tipo especial de función en JavaScript que puede **pausar** y **reanudar** su ejecución. Esto las hace útiles cuando queremos manejar iteraciones de una manera más eficiente.

En lugar de ejecutar todo el código de una vez, una función generadora devuelve

valores **uno por uno** usando `yield`. Cuando la función se pausa con `yield`, recuerda su estado hasta que la volvamos a llamar con `next()`.

🔹 **Ejemplo:**

```js
function* contador() {
  let i = 1;
  while (true) {
    yield i++; // Pausa la ejecución y devuelve i
  }
}

const generador = contador();
console.log(generador.next().value); // 1
console.log(generador.next().value); // 2
console.log(generador.next().value); // 3
```

💡 **¿Para qué sirven?**
Se usan para **iteraciones personalizadas**, manejar **datos en streaming** y evitar **bloqueos en operaciones costosas**.

🔹 Explica el alcance y la cadena de alcance en JavaScript

El **alcance (scope)** en JavaScript determina **dónde** una variable puede ser accedida. La **cadena de alcance (scope chain)** permite a una función acceder a variables de su contexto externo.

🔹 **Tipos de Scope en JavaScript:**

1. **Global** → Variables accesibles en cualquier parte del código.
2. **De Función** → Variables accesibles solo dentro de una función.
3. **De Bloque** → Variables declaradas con `let` y `const` solo existen dentro del bloque `{}`.

🔹 **Ejemplo de Scope Chain:**

```js
const global = "Soy global";

function externa() {
  const externaVar = "Soy externa";
```

```
  function interna() {
    console.log(global); // "Soy global"
    console.log(externaVar); // "Soy externa"
  }

  interna();
}

externa();
```

💡 **Si una función no encuentra una variable en su propio scope, buscará en el scope superior hasta llegar al global.**

🔹 ¿Qué son las devoluciones de llamada (Callbacks) en JavaScript?

Un **callback** es una función que se pasa como argumento a otra función y se ejecuta después de que la tarea principal ha terminado. Se usa mucho en operaciones **asíncronas**, como trabajar con datos de una API o temporizadores.

🔹 **Ejemplo de un Callback básico:**

```js
function procesar(entrada, callback) {
  console.log("Procesando datos...");
  setTimeout(() => callback(`Resultado: ${entrada * 2}`), 2000);
}

procesar(5, (resultado) => {
  console.log(resultado); // "Resultado: 10" (después de 2 segundos)
});
```

💡 **Problema:** Si encadenamos muchos callbacks, el código se vuelve difícil de leer y mantener (callback hell). Esto se soluciona con **promesas y async/await**.

🔹 ¿Cuál es la diferencia entre declaración de función y expresión de función?

En JavaScript hay dos formas principales de definir funciones:

Tipo	Ejemplo	¿Soporta Hoisting?

| **Declaración de función** | `function suma(a, b) { return a + b; }` | ✅ Sí |
| **Expresión de función** | `const suma = function(a, b) { return a + b; };` | ❌ No |

🔹 **Ejemplo de diferencia en Hoisting:**

```js
saludar(); // ✅ Funciona porque fue "elevada"
function saludar() {
  console.log("Hola");
}
```

```js
saludar(); // ❌ Error: Cannot access 'saludar' before initialization
const saludar = function() {
  console.log("Hola");
};
```

💡 **Las funciones declaradas se "mueven" arriba del código (Hoisting), mientras que las expresiones no.**

🔹 ¿Qué son las importaciones y exportaciones en JavaScript?

JavaScript permite dividir código en **módulos**, donde un archivo puede exportar funciones y otro importarlas.

🔹 **Ejemplo de Exportación (archivo `math.js`):**

```js
export const suma = (a, b) => a + b;
export default function multiplica(a, b) { return a * b; }
```

🔹 **Ejemplo de Importación (archivo `app.js`):**

```js
import { suma } from "./math.js";
import multiplica from "./math.js";

console.log(suma(3, 4)); // 7
console.log(multiplica(3, 4)); // 12
```

💡 **Las exportaciones nombradas necesitan llaves `{}`, mientras que las por defecto no.**

🔹 Explica ¿Qué es DOM y cómo funciona?

El **DOM (Document Object Model)** es la representación de una página web en forma de árbol de objetos. Nos permite modificar contenido HTML con JavaScript.

🔹 **Ejemplo de manipulación del DOM:**

```js
document.getElementById("titulo").textContent = "Nuevo Título";
document.querySelector(".btn").addEventListener("click", () => alert("¡Hola!"));
```

💡 **Cada etiqueta HTML es un nodo en el DOM, y podemos modificarlo con JavaScript.**

🔹 ¿Qué es el parámetro rest y el operador spread?

Operador	Uso	Ejemplo
`rest` (`...`)	Agrupa múltiples argumentos en un array	`function suma(...nums) {}`
`spread` (`...`)	Expande los elementos de un array u objeto	`const nums = [1, 2, 3]; console.log(...nums);`

🔹 **Ejemplo de Rest:**

```js
function suma(...numeros) {
  return numeros.reduce((acc, num) => acc + num, 0);
}
console.log(suma(1, 2, 3, 4)); // 10
```

🔹 **Ejemplo de Spread:**

```js
const arr = [1, 2, 3];
const nuevoArr = [...arr, 4, 5];
```

```
console.log(nuevoArr); // [1, 2, 3, 4, 5]
```

💡 **Rest agrupa valores, Spread los expande.**

🔹 ¿Cuál es la diferencia entre `Undefined`, `Undeclared` y `Null` en JavaScript?

Estado	Significado	Ejemplo
Undefined	Variable declarada pero sin valor asignado	`let x; console.log(x); // undefined`
Undeclared	Variable nunca declarada	`console.log(y); // ReferenceError`
Null	Valor asignado manualmente para indicar "sin valor"	`let z = null; console.log(z); // null`

💡 **Ejemplo visual:**

```js
let a;
console.log(a); // undefined

console.log(b); // ❌ ReferenceError (b no fue declarada)

let c = null;
console.log(c); // null
```

💡 **Undefined es el estado predeterminado, Null es intencional y Undeclared es un error.**

📚 Recursos adicionales

📖 **Documentación Oficial**:

- [MDN JavaScript Docs](https://developer.mozilla.org/es/docs/Web/JavaScript)
- [JavaScript.info](https://javascript.info/)

🎥 **Videos Recomendados**:

- [JavaScript Crash Course - Traversy Media](https://youtu.be/hdI2bqOjy3c)
- [Event Loop Explained - Philip Roberts](https://www.youtube.com/watch?

```
v=8aGhZQkoFbQ)

🔗 **Cursos Recomendados**:

- [JavaScript: The Good Parts - Udemy](https://www.udemy.com/course/the-complete-
javascript-course/)
```